GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 2
CLIENT 2
INSTRUCTIONS ..... ERROR! BOOKMARK NOT DEFINED.
DEVELOPER 3
1. ALGORITHM CIPHER 3
2. CERTIFICATE GENERATION 3
3. DEPLOY CIPHER 3
4. SECURE COMMUNICATIONS 4
5. SECONDARY TESTING 4
6. FUNCTIONAL TESTING 4
7. SUMMARY 5
8. INDUSTRY STANDARD BEST PRACTICES 5

Document Revision History

Version	Date	Author	Comments
1.0	February 24 2026	Zander Taylor	

Client

ARTEMIS FINANCIAL

Developer
[Zander Taylor]

1. Algorithm Cipher

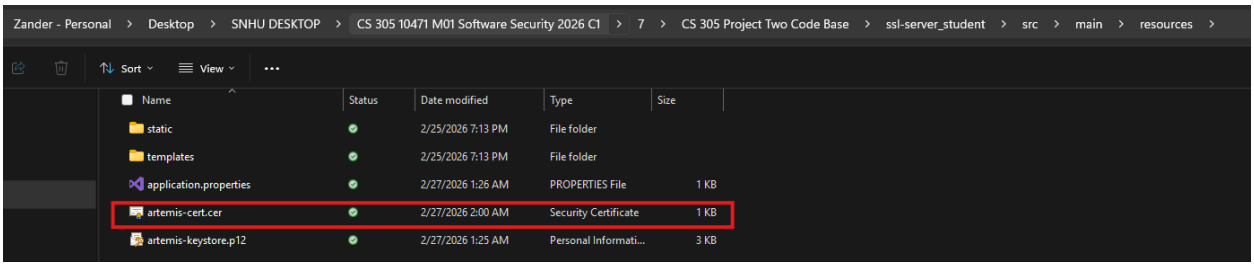
Artemis Financial required modern cryptographic protections to ensure confidentiality, integrity, and secure communications across its public web interface. To meet these requirements, the SHA-256 cryptographic hash algorithm and RSA 2048 asymmetric encryption were implemented as part of the security enhancements.

SHA-256 is a member of the Secure Hash Algorithm 2 family and produces a 256-bit fixed-length digest. Hash functions are one-way mathematical operations designed to ensure data integrity. Unlike encryption, hashing does not allow the original data to be reconstructed. Instead, it generates a unique digital fingerprint of the input data. In this implementation, SHA-256 was selected because it is resistant to collision attacks and is currently approved for secure federal and industry use. The 256-bit output provides strong resistance against brute force attacks and ensures that even minor changes to the input string produce significantly different hash outputs.

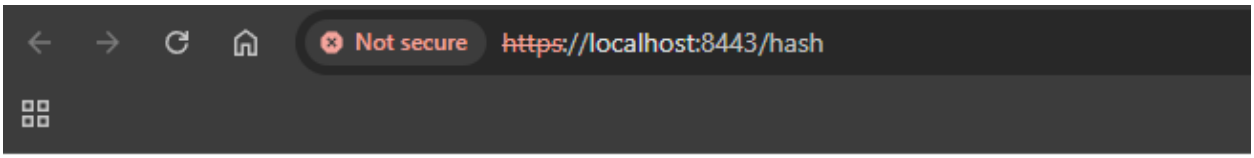
For secure communications, RSA 2048-bit asymmetric encryption was used within a self-signed certificate deployed through a PKCS12 keystore. RSA is a public key cryptosystem that uses mathematically related key pairs: a public key for encryption and a private key for decryption. The 2048-bit key size provides strong cryptographic security and meets modern industry recommendations. Random number generation plays a critical role in key creation, as secure key pairs rely on high entropy values to prevent predictability. The TLS protocol uses this asymmetric key exchange to establish a secure session and then negotiates symmetric session keys for efficient encrypted communication.

Historically, earlier algorithms such as MD5 and SHA-1 were widely used but are now considered cryptographically weak due to demonstrated collision vulnerabilities. Modern security standards recommend SHA-256 or stronger hash functions and 2048-bit or greater asymmetric keys. By implementing SHA-256 for checksum verification and RSA 2048 within TLS, Artemis Financial's application now aligns with contemporary encryption standards and industry best practices for secure software development.

2. Certificate Generation

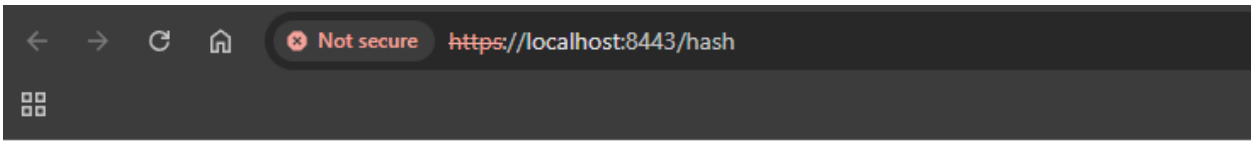


3. Deploy Cipher



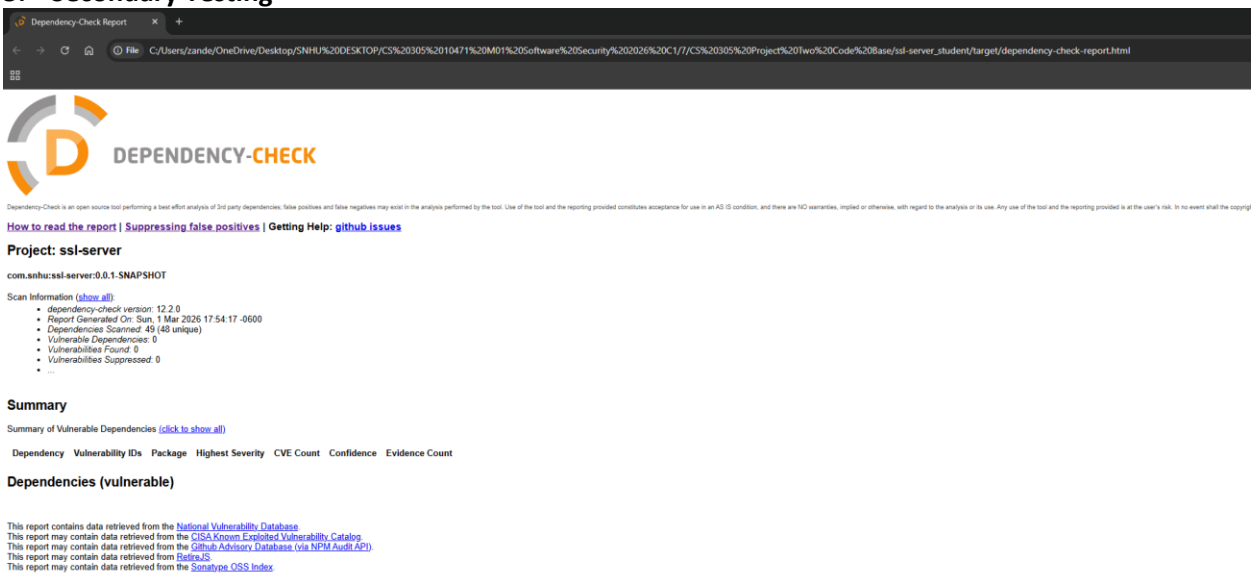
Original Data: Zander Taylor - CS305 - 2026 Secure Checksum Test
SHA-256 Checksum: b08fefa2972ca2fb889c5d20991e16837fe15dc0691d2b8cb6e36844f555ded6

4. Secure Communications

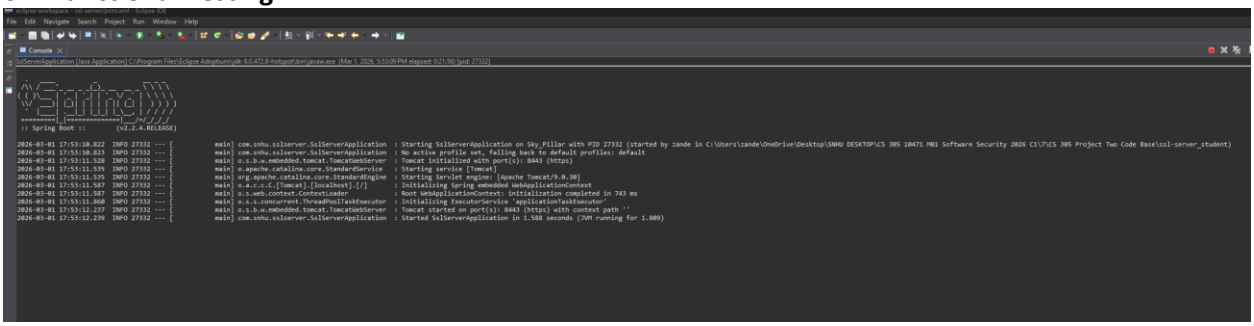


Original Data: Zander Taylor - CS305 - 2026 Secure Checksum Test
SHA-256 Checksum: b08fefa2972ca2fb889c5d20991e16837fe15dc0691d2b8cb6e36844f555ded6

5. Secondary Testing



6. Functional Testing



7. Summary

I refactored the application to strengthen transport security, improve data integrity validation, and incorporate automated vulnerability assessment into the development lifecycle. The primary refactoring change involved converting HTTP communication to HTTPS using a PKCS12 keystore and SSL certificate configuration within the application properties file. This ensured that all client server communication is encrypted using TLS, protecting data in transit from interception or tampering. Additionally, I implemented a REST controller endpoint to generate a SHA-256 checksum. This introduced a cryptographic integrity mechanism that demonstrates secure hashing practices and reinforces protection against data manipulation at the same time.

Referring to the vulnerability assessment process flow diagram, the following security areas were addressed during refactoring:

- Identification: Third party dependencies were analyzed using OWASP Dependency Check to identify known CVEs.
- Assessment: Application configuration and communication protocols were reviewed for insecure defaults such as unencrypted HTTP.
- Mitigation: HTTPS was enforced and SSL/TLS configuration was properly implemented to eliminate insecure transport.
- Validation: Functional testing confirmed successful application execution without runtime errors, and dependency scanning verified that no new vulnerabilities were introduced.

Layers of security were added through a defense in depth approach:

1. Transport Layer Security via HTTPS encryption.
2. Certificate-based trust configuration using a keystore.
3. Cryptographic hashing using SHA-256 for integrity validation.
4. Automated dependency vulnerability scanning integrated into the Maven build lifecycle.
5. Functional execution testing to ensure secure refactoring did not break application behavior.

By combining encrypted communication, cryptographic validation, configuration hardening, and automated vulnerability analysis, this newly refactored application complies with established security testing protocols and aligns with secure software development lifecycle practices.

8. Industry Standard Best Practices

I implemented various industry standard best practices for secure coding throughout the refactoring process to mitigate known security vulnerabilities and strengthen the application's overall security posture.

First, HTTPS was enforced using TLS encryption through proper SSL configuration and a PKCS12 keystore. Enforcing encrypted communication aligns with industry standards such as OWASP recommendations and NIST guidance for protecting data in transit. Removing unsecured HTTP eliminates exposure to man in the middle attacks and interception risks.

Second, SHA-256 was used for checksum generation. SHA-256 is a widely accepted cryptographic hashing algorithm recommended for integrity verification. Using a strong, modern hashing standard avoids deprecated or insecure algorithms such as MD5 or SHA-1, which are both vulnerable to collision attacks.

Third, automated dependency vulnerability scanning was implemented using OWASP Dependency Check within the Maven build lifecycle. This reflects industry standard DevSecOps practices by integrating security analysis into the development pipeline rather than treating security it as a separate, post development activity. Automated scanning helps identify known CVEs in third-party libraries before deployment.

To maintain the application's existing security:

- Refactored avoiding introducing insecure libraries or deprecated APIs.
- SSL configuration was implemented using secure defaults rather than custom cryptographic implementations.
- No hard coded credentials or secrets were introduced into the source code.
- Functional testing verified that security enhancements did not degrade stability or introduce any logic errors.
- Dependency analysis confirmed that no additional vulnerabilities were introduced during refactoring.

The value of applying industry standard secure coding practices to the company's overall well being is substantial. Secure coding reduces the likelihood of data breaches, protects sensitive customer and organizational data, and minimizes any legal and regulatory risk exposure. Proactive vulnerability management lowers long term remediation costs and prevents reputational damage associated with the weight of security incidents.

Following this way of thinking, integrating automated security analysis into the build process promotes a culture of continuous security validation. This ensures that future changes are evaluated against evolving threat intelligence, supporting sustainable and scalable security practices.
By applying encrypted transport, secure cryptographic standards, automated vulnerability scanning, and controlled configuration management, the refactored application aligns with recognized secure software development principles and contributes to the organization's operational resilience and risk reduction strategy.